

Milestone-1 Report

NLP Foundation & Semantic Similarity: A TF-IDF Baseline for Multiple-Choice Question Answering

Naini Diwan | Roll No. 23f3001480

Smart MCQ Solver Challenge (Kaggle Competition)

1. Introduction

A baseline system was built to solve multiple-choice questions (MCQs) using classical NLP techniques, developed for the Smart MCQ Solver Challenge. Each record in the dataset consists of a question prompt and five candidate answers (A through E), with the task being to identify the correct option. The dataset was sourced from the private Kaggle competition smart-mcq-solver-challenge hosted by IIT Madras. Dataset comprised of train.csv and test.csv files, with answers encoded as single letters mapped to integer labels (A→0, B→1, C→2, D→3, E→4).

The approach taken here treats answer selection as a semantic similarity problem: rather than training a classifier, the prompt and each of its five options are converted into numerical vectors, and the option whose vector is most similar to the prompt's vector is selected as the predicted answer. This is a reasonable first baseline because it requires no labeled training beyond fitting a vocabulary, and it gives a lower bound against which more sophisticated models (transformers, fine-tuned classifiers) can be compared.

2. Data Preprocessing

Before vectorization, prompt text was cleaned in two steps.

Case normalization. All prompt text was lowercased using a straightforward `.apply(lambda x: x.lower())` call on the prompt column. This reduces vocabulary sparsity by treating "Heidegger" and "heidegger" as the same token, which matters for a bag-of-words style model where every distinct string is a separate feature.

Punctuation handling. Using Python's `string.punctuation` set, punctuation characters were stripped from the prompt text, with two explicit exceptions: question marks (?) and colons (:). This is a deliberate choice rather than a blanket strip. A check confirmed that every single prompt in the training set contains a ? or : character (a count of rows without either returned zero), meaning these characters likely carry structural signal — for instance, marking where an instruction ("identify the correct statement:") hands off to the actual question. Removing them indiscriminately could have erased that structure.

Notably, the answer options (columns A–E) were left unprocessed in their original casing and punctuation. This is a modeling decision worth flagging: TF-IDF vectors for prompts and options are built from the same fitted vocabulary, but only one side was normalized. This asymmetry could reduce matching quality, since the same word (e.g., "Blueshifting" in an option vs. a hypothetical lowercase equivalent in the prompt) may not vectorize identically.

3. Model: TF-IDF and Cosine Similarity

Why TF-IDF over CountVectorizer. A raw count-based vectorizer treats every word as equally informative, which lets frequent but low-information words (like "the," "is," "what") dominate the vector. TF-IDF corrects for this: it downweights terms that appear across many documents in the corpus and upweights terms that are distinctive to a smaller subset. Since MCQ prompts share heavy structural overlap ("select the most accurate option," "identify the correct statement"), TF-IDF's

inverse-document-frequency term is what lets content words (the actual subject matter of the question) carry more weight than the templated instruction language.

The vectorizer was configured with `max_df=0.95`, meaning any term appearing in more than 95% of documents is dropped entirely from the vocabulary — a further guard against boilerplate phrasing swamping the signal.

Why cosine similarity over Euclidean distance. Cosine similarity measures the angle between two vectors, not their magnitude. Two texts about the same topic can produce vectors of very different lengths simply because one is longer, uses more repeated terms, or has a larger vocabulary footprint. Euclidean distance is sensitive to this length difference; cosine similarity is not, since it normalizes by vector magnitude. This makes it the more appropriate metric here, where prompts and their candidate answers are frequently different lengths.

Vocabulary construction. To avoid data leakage, the vectorizer was fit only on the training split. The fitting corpus combined every prompt in the training set with every value from all five option columns (A–E), giving the vectorizer visibility into the full range of vocabulary it would need to encode at inference time.

Train/validation split. The data was split 80/20 using `train_test_split` with `stratify=df['answer']`, ensuring the distribution of correct-answer letters (A–E) was preserved in both splits. A fixed random seed (42) was set across `random`, `numpy`, and the `PYTHONHASHSEED` environment variable for reproducibility.

Inference procedure. For each validation row, the prompt was transformed into a TF-IDF vector, and each of the five options was transformed the same way. Cosine similarity was computed between the prompt vector and each of the five option vectors, producing five similarity scores per question. These scores serve as a proxy for a model's "confidence" in each option, standing in for the logits a trained classifier would normally output.

4. Evaluation Methodology

Three metrics were used to assess performance, each capturing a different notion of correctness.

Accuracy. The proportion of questions where the top-ranked option (highest cosine similarity) matches the true answer. This is the strictest metric — it only credits an exact top-1 match.

F1 score (macro-averaged). Rather than a single accuracy figure, macro F1 computes precision and recall for each answer class (A through E) independently, then averages them without weighting by class frequency. This matters because a model could get high accuracy purely by favoring one or two heavily represented answer letters — macro F1 penalizes that kind of imbalance and gives a fairer picture of whether the model performs consistently across all five options.

MAP@3 (Mean Average Precision at 3). This metric gives partial credit: instead of requiring the correct answer to be ranked first, it checks whether the correct answer appears anywhere in the top 3 ranked predictions, and rewards it more if it appears earlier. Concretely, for each question, if the true answer is found at rank 1, 2, or 3, the score contribution is 1, 1/2, or 1/3 respectively; if it doesn't appear in the top 3, the contribution is 0. These per-question scores are averaged across the full validation set to produce MAP@3. The final metric is bounded between 0 (no correct answers ever in the top 3) and 1 (correct answer always ranked first).

The value of MAP@3 in this context is that it distinguishes between a model that is "close but not quite" — ranking the right answer second or third — and one that is missing the signal entirely. A pure top-1 accuracy score cannot make that distinction, which matters when deciding whether a model's failures stem from a genuinely wrong understanding of the question versus fine-grained ranking noise among similar options.

The evaluation logic was implemented twice in the notebook: once as a general-purpose `calculate_metrics` function designed to accept model logits and labels (anticipating future use with a trained classifier), and once as an inline computation applied directly to the TF-IDF cosine similarity scores for the validation set.

5. Results

The TF-IDF baseline was evaluated on the held-out validation split (20% of the training data), yielding the following scores:

Metric	Score
Accuracy	0.1275
F1 Score (macro)	0.1268
MAP@3	0.2583

These numbers are notably weak. With five answer options per question, random guessing would be expected to produce roughly 0.20 accuracy. The TF-IDF baseline's accuracy of 0.1275 falls *below* that random-guess threshold, and its macro F1 tracks closely alongside it, indicating the model is not just weak but actively performing worse than chance at ranking the correct option first.

MAP@3, at 0.2583, is higher than the top-1 metrics, which makes sense given it credits correct answers found in positions 2 or 3, not just 1. But even this more forgiving metric confirms the model struggles to consistently locate the right answer within its top three guesses.

6. Discussion

The underperformance here is informative rather than simply disappointing, and points to a mismatch between the technique and the task. TF-IDF and cosine similarity work by rewarding lexical overlap between two pieces of text — they detect when a prompt and an option share the same words. But looking at the sample data (the Heidegger and simultaneity questions, for example), the correct answer is not the option that repeats the most words from the prompt. It's the option that is *semantically* correct, often phrased in ways that share very little vocabulary with the question itself. In several visible examples, multiple wrong options are near-paraphrases of each other and of the correct answer, differing only in a negation or a swapped noun ("humans exist within time" vs. "humans do not exist within time"), which is exactly the kind of distinction bag-of-words vectorization cannot capture, since both phrasings produce nearly identical TF-IDF vectors.

This explains why accuracy underperforms random guessing: cosine similarity is essentially blind to negation and to fine semantic contrasts, so when options are lexically similar but meaningfully

opposite, the model has no reliable signal to distinguish them, and may even be systematically misled toward wrong answers that happen to share more incidental vocabulary with the prompt's instructional framing.

A secondary factor worth noting is the preprocessing asymmetry described in Section 2: prompts were lowercased and selectively depunctuated, while options were not. Any case or punctuation mismatch between a prompt's vocabulary and an option's vocabulary could suppress similarity scores independent of actual semantic relevance.

7. Conclusion and Next Steps

This baseline confirms that lexical-overlap methods like TF-IDF with cosine similarity are insufficient for this MCQ task, where correctness hinges on meaning rather than word overlap. The exercise was still valuable: it established a clear, reproducible floor (accuracy 0.1275, F1 0.1268, MAP@3 0.2583) against which more capable approaches can be measured, and it surfaced a concrete diagnosis (negation-blindness, lexical-similarity-without-semantic-similarity) that should directly inform the next modeling step.

Natural next steps include applying dense transformers in place of TF-IDF, which encode meaning rather than surface word overlap and would likely handle negation and paraphrase far better; fine-tuning a transformer classifier directly on the prompt-option pairs; and correcting the preprocessing asymmetry so prompts and options are normalized identically before any further baseline comparisons are drawn.